

Documentation – Station de remplissage

1. Présentation du projet

Nom : Station de remplissage

Version : 1.1.0

Technologie : Python + PyQt5 + MySQL

Objectif

Cette application permet de :

- Gérer un système de remplissage
 - Scanner des QR codes
 - Enregistrer les données dans une base MySQL
 - Suivre les actions via un système de logs
 - Afficher une interface utilisateur fluide avec animations
-

2. Architecture du projet

Structure des fichiers

```
project/
├── main.py                # Application principale (UI + logique)
├── mysql_manager.py       # Gestion de la base de données MySQL
├── logger.py              # Système de logs
├── style.py               # Styles UI (CSS PyQt)
├── install.py             # Installation / configuration
├── start.bat              # Lancement rapide (Windows)
├── assets/                # Ressources graphiques et sons
│   ├── logo.png
│   ├── spinner.gif
│   ├── start.wav
│   └── close.wav
└── logs/                  # Fichiers de logs journaliers
```

3. ⚙️ Fonctionnement global

🔄 Cycle principal

1. L'application démarre (`main.py`)
 2. Connexion à la base MySQL
 3. Affichage de l'interface utilisateur
 4. Scan d'un QR code
 5. Traitement des données :
 - Vérification
 - Mise à jour BDD
 - Incrémentation compteur
 6. Affichage animation de remplissage
 7. Log des actions
-

4. 📱 □ Interface utilisateur (PyQt5)

□ Composants principaux

- `QWidget` → Fenêtre principale
 - `QStackedLayout` → Gestion des pages
 - `QLabel` → Affichage (texte/images)
 - `QMovie` → Animation (GIF)
 - `QSoundEffect` → Sons
 - Styles personnalisés via `style.py`
-

6. 🗄️ Système de logs

✦ Fonction principale

```
log(action, details)
```

📁 Emplacement

```
logs/YYYY-MM-DD.log
```

📄 Exemple

```
[14:32:10] MYSQL | Connexion OK  
[14:32:15] SCAN | QR détecté : 12345
```

✓ Utilité

- Debug
 - Traçabilité
 - Audit système
-

7. 🐛 Panneau de debug (fonction cachée)

🔍 Description

Le projet intègre un **panneau de debug caché**, activable via une interaction spécifique.

Variables associées

```
self.debug_visible = False  
self.debug_counter = 0
```

🎯 Objectifs du panneau debug

- Voir les informations internes en temps réel
 - Tester sans redémarrer l'application
 - Diagnostiquer les bugs rapidement
-

⚙ Fonctionnement :

Le panneau est activé via :

- Une série d'appuis sur la touche **D**

🔗 Le compteur `debug_counter` sert à détecter cette action.

📄 Informations affichées

Le panneau contient :

- Un menu permettant d'ajouter des qr codes avec un ID (chiffre)
-

🔒 Activation (logique typique)

Exemple de logique possible :

```
self.debug_counter += 1

if self.debug_counter >= 5:
    self.debug_visible = not self.debug_visible
    self.debug_counter = 0
```

📋 Utilisation du debug

Cas pratiques

- ✖ QR non reconnu → vérifier `last_qr`
 - ✖ BDD ne répond pas → vérifier état MySQL
 - ✖ UI bloquée → vérifier logs
 - ✖ Bugs intermittents → surveiller en live
-

8. 🔊 Gestion des sons

- Son de démarrage : `start.wav`
- Son de fermeture : `close.wav`

Lecture via :

`QSoundEffect`

9. 🎨 Styles

Gérés dans :

`style.py`

Permet :

- Uniformité visuelle
 - Maintenance simplifiée
 - Thème personnalisable
-

10. 🚀 Lancement du projet

Méthode 1

```
python install.py (obligatoire pour installer les dépendances et sur les 2 méthodes)
python main.py
```

Méthode 2 (Windows)

`start.bat`

11. ⚠️ Points d'amélioration

- 🔒 Sécuriser les identifiants MySQL
 - 🏠 Ajouter interface admin
 - 🌐 Ajouter API ou backend distant
 - ☐ Ajouter tests automatisés
 - 📦 Packager en .exe
-

12. □ Résumé

Ce projet est une application complète avec :

- Interface graphique moderne (PyQt5)
 - Gestion QR
 - Mode debug caché
-